

Методы снижения размерности

Ефимов Владислав

 образование

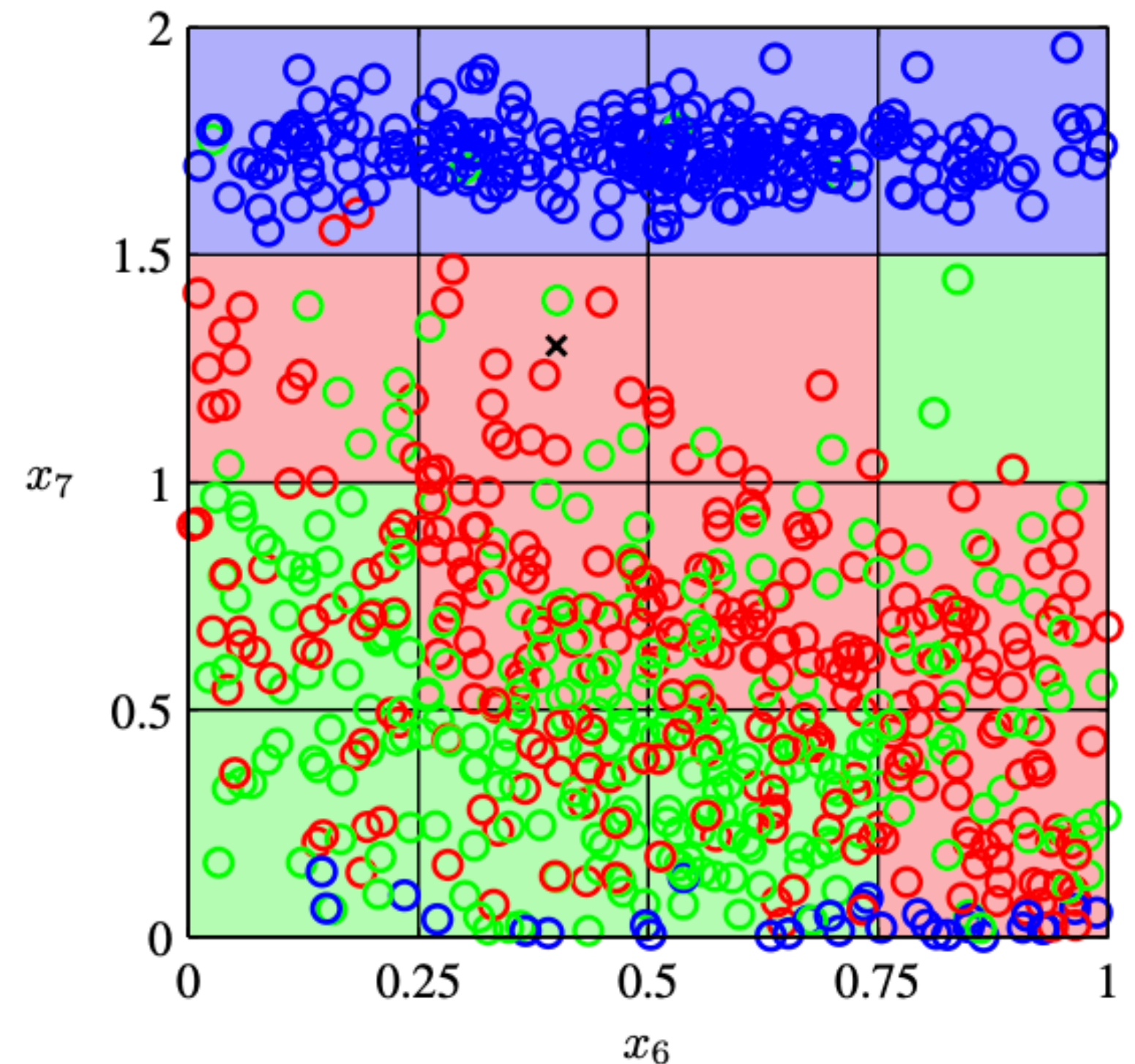
Проклятие размерности



Проклятие размерности

Проблема, связанная с
экспоненциальным возрастанием
количества данных из-за увеличения
размерности пространства

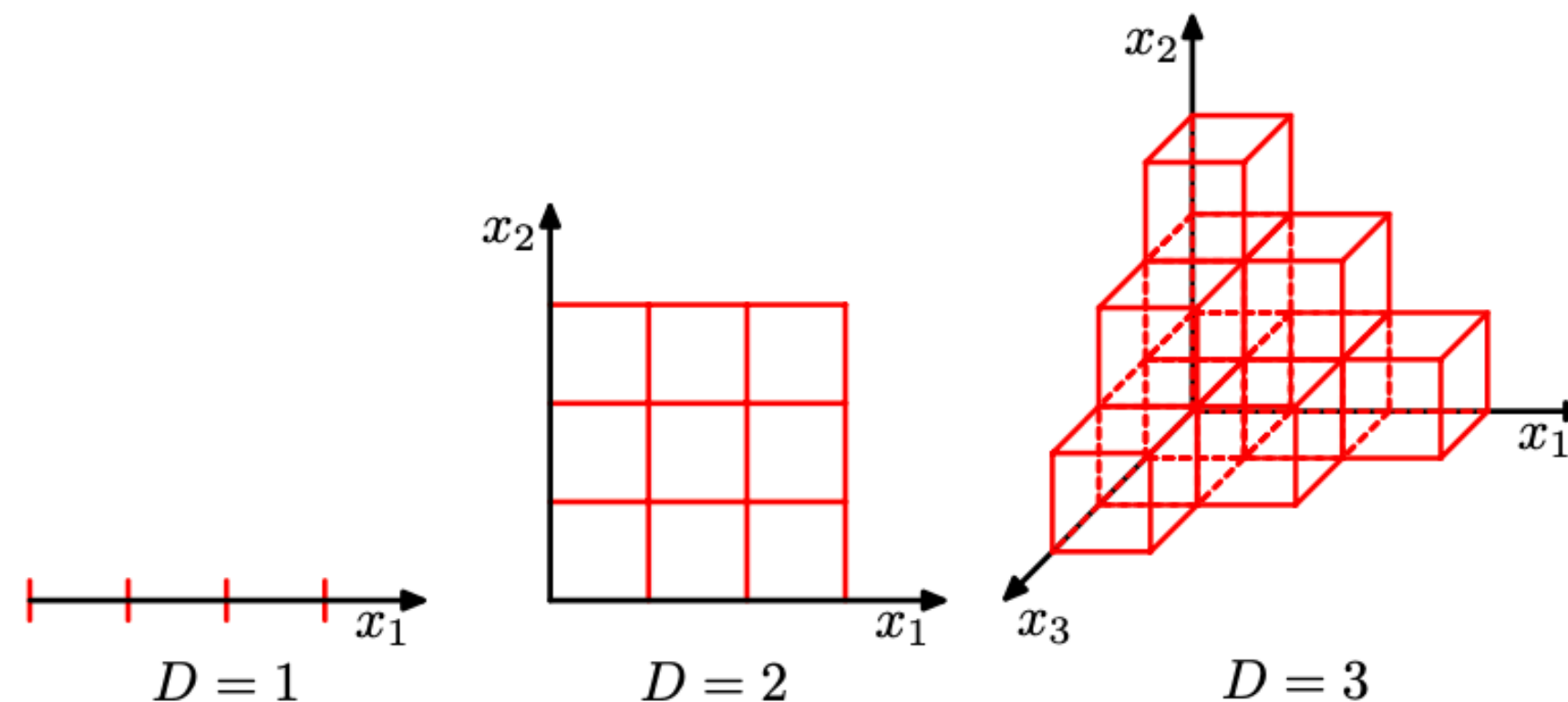
Рассмотри пример с задачей
классификации.



Пример

При увеличении размерности пространства для равномерного покрытия многомерного пространства требуется **экспоненциальное** увеличение количества данных

Хотим равномерно покрыть n -мерный единичный куб точками так, чтобы хотя бы одна точка была внутри каждого куба.



Почему это проблема

... а также

- повышается трудоемкость вычислений
- необходимость хранения большого количества данных
- увеличение доли шумов
- в линейных классификаторах приводит к мультиколлинеарности и переобучению

Почему это проблема

$$x_1 = (a_1, a_2, \dots, a_n) \qquad x_2 = (a_1, a_2 + \Delta, \dots, a_N)$$

$$x_3 = (a_1 + \epsilon, a_2 + \epsilon, \dots, a_N + \epsilon), \epsilon \ll \Delta$$

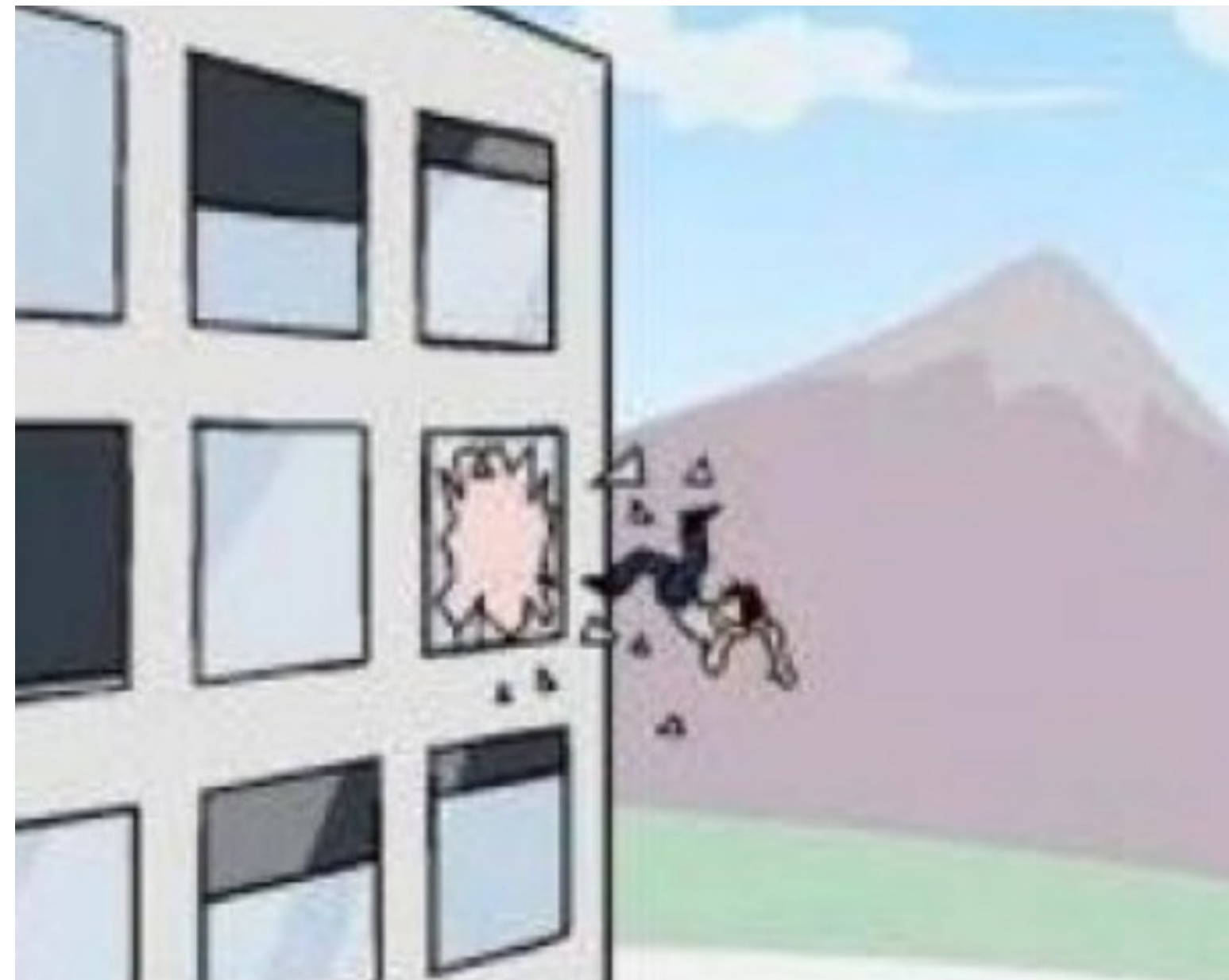
В пространстве высокой размерности $\rho(x_1, x_2) \approx \rho(x_1, x_3)$

То есть небольшие изменения ϵ в большом количестве координат вызывают сравнимые изменения в расстоянии, как и значительное изменение Δ в 1 координате

Расстояния во всех парах объектов стремятся (согласно ЗБЧ) к одному и тому же значению и становятся неинформативными - проблема для метрических классификаторов

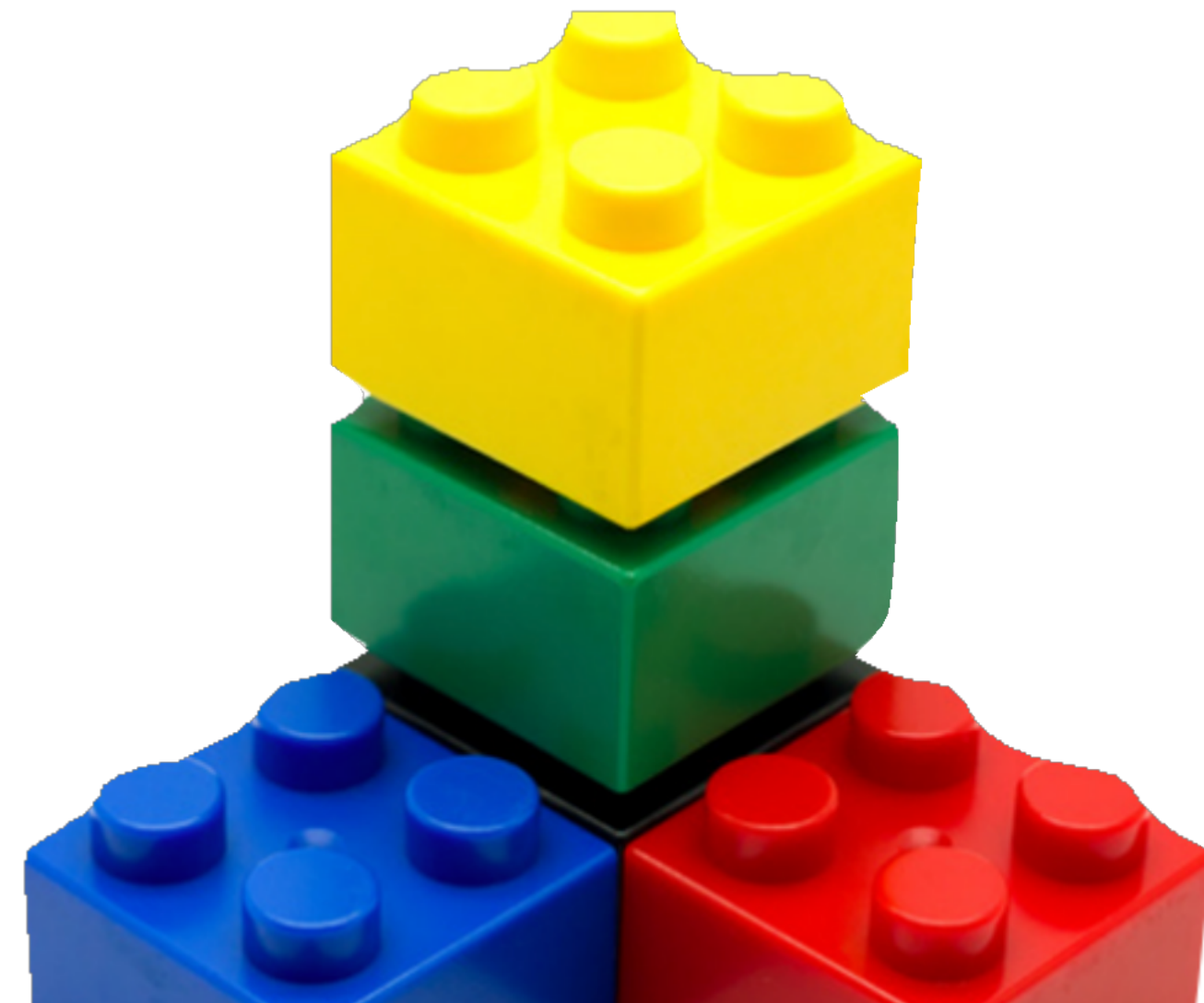
Как устранить - уменьшить количество признаков

- Выбросить часть признаков



Как устранить - уменьшить количество признаков

- Выбросить часть признаков
- Построить меньшее количество признаков на основе старых



Отбор признаков



Зачем отбирать

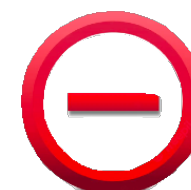
- боремся с проклятием размерности
- могут быть шумовые или коррелирующие признаки
- ограничение на ресурсы (модель/данные не влезают в память)
- ускорение обучения/инференса



Одномерный отбор

- оцениваем информативность каждого признака отдельно
- отбираем k лучших или выше порога

 просто

 не учитывается *взаимодействие* признаков между собой
(пропустим коллинеарные признаки)

Информативность признака

- корреляция/РМІ с таргетом

$$\text{pmi}(x; y) \equiv \log \frac{p(x, y)}{p(x)p(y)}$$

Информативность признака

- корреляция/PMI с таргетом
- метрики классификатора (на 1 признаке/деревья)

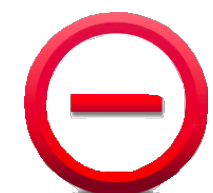
Информативность признака

- корреляция/PMI с таргетом
- метрики классификатора (на 1 признаке/деревья)
- статистические критерии (chi-square, ...)

Жадные методы

- Перебираем комбинации (переборные методы, жадные методы, ADD-DEL)
- Для каждой комбинации обучаем модель
- Смотрим метрики на валидации
- Берем набор признаков с лучшим качеством

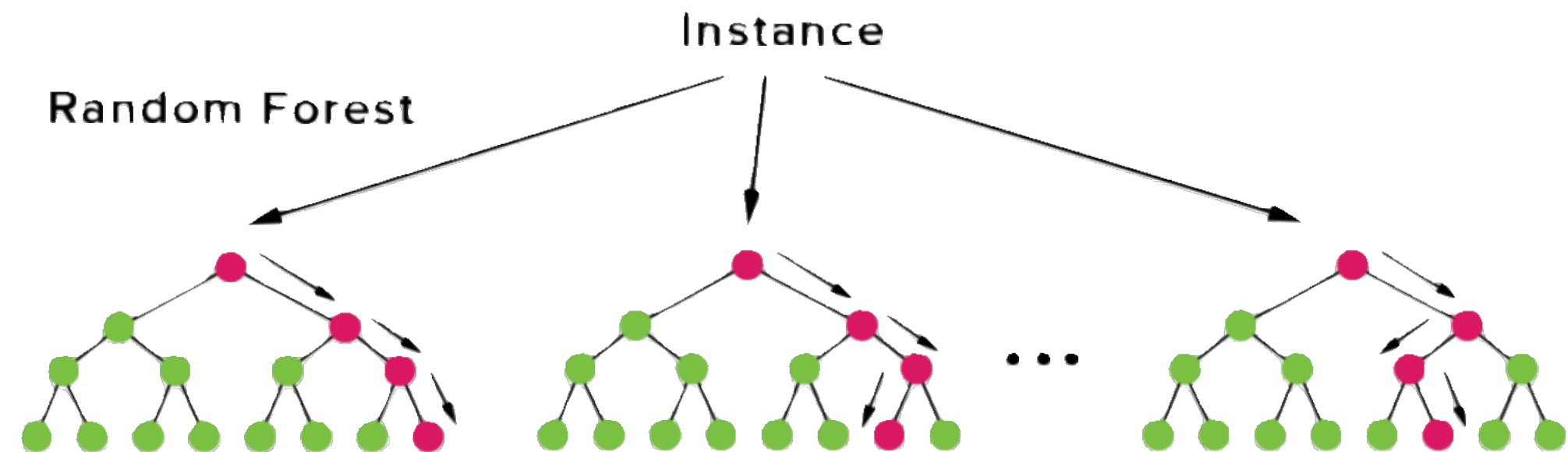
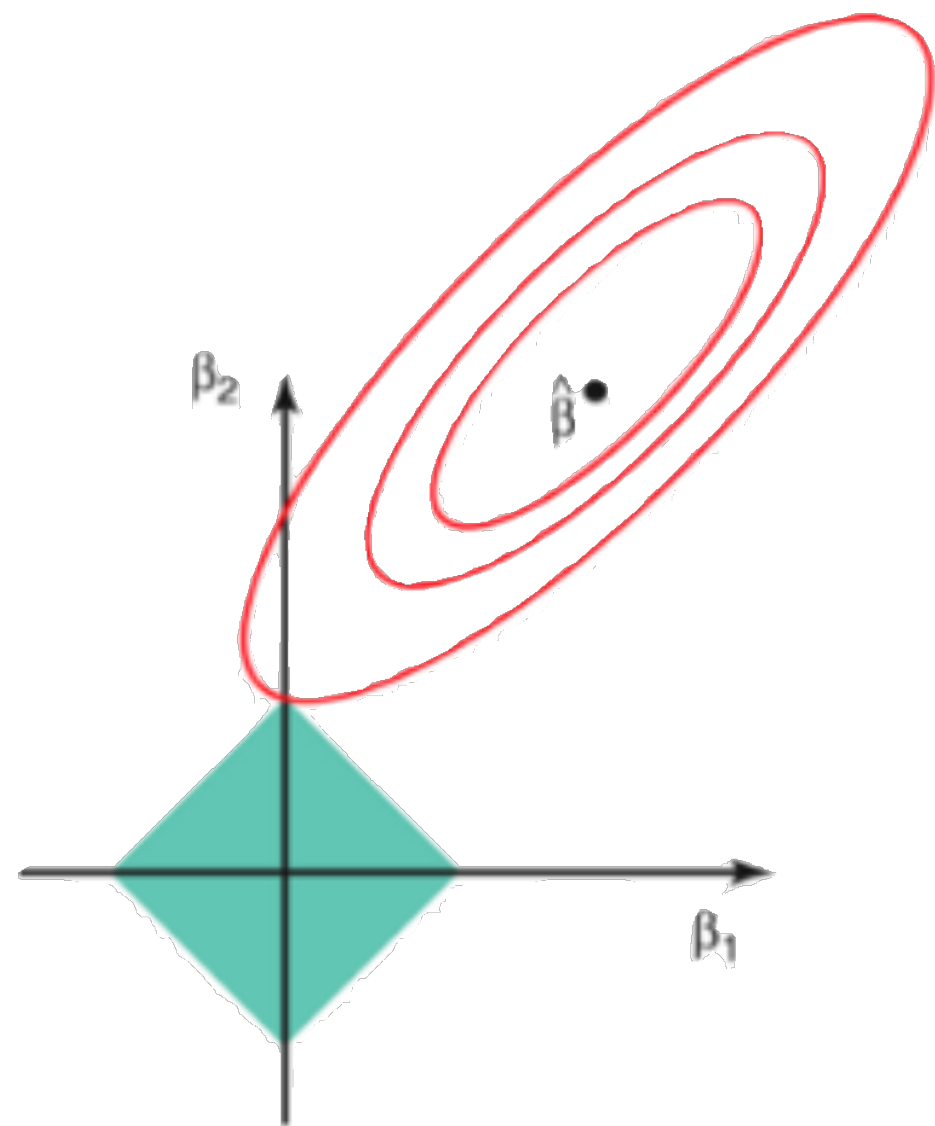
 учитывает *взаимодействие* признаков

 затратно (много комбинаций, обучение на каждом наборе)

 жадные методы иногда слишком жадные $\neg(\cup)$

Отбор на основе моделей

- Линейные модели (веса, как показатель информативности)
- L1-регуляризация
- Деревья, случайный лес, бустинг над деревьями
(изменение критерия информативности)



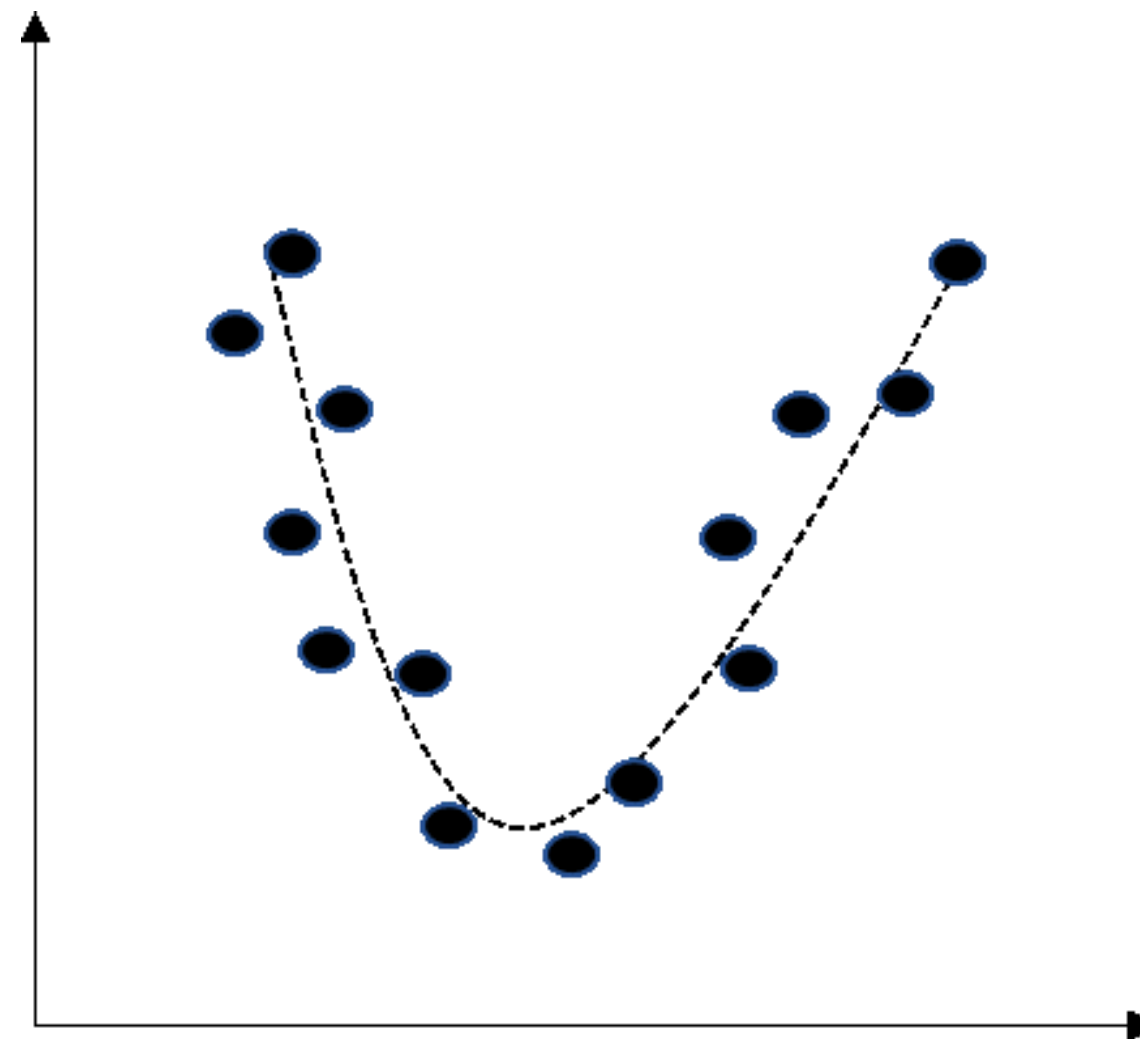
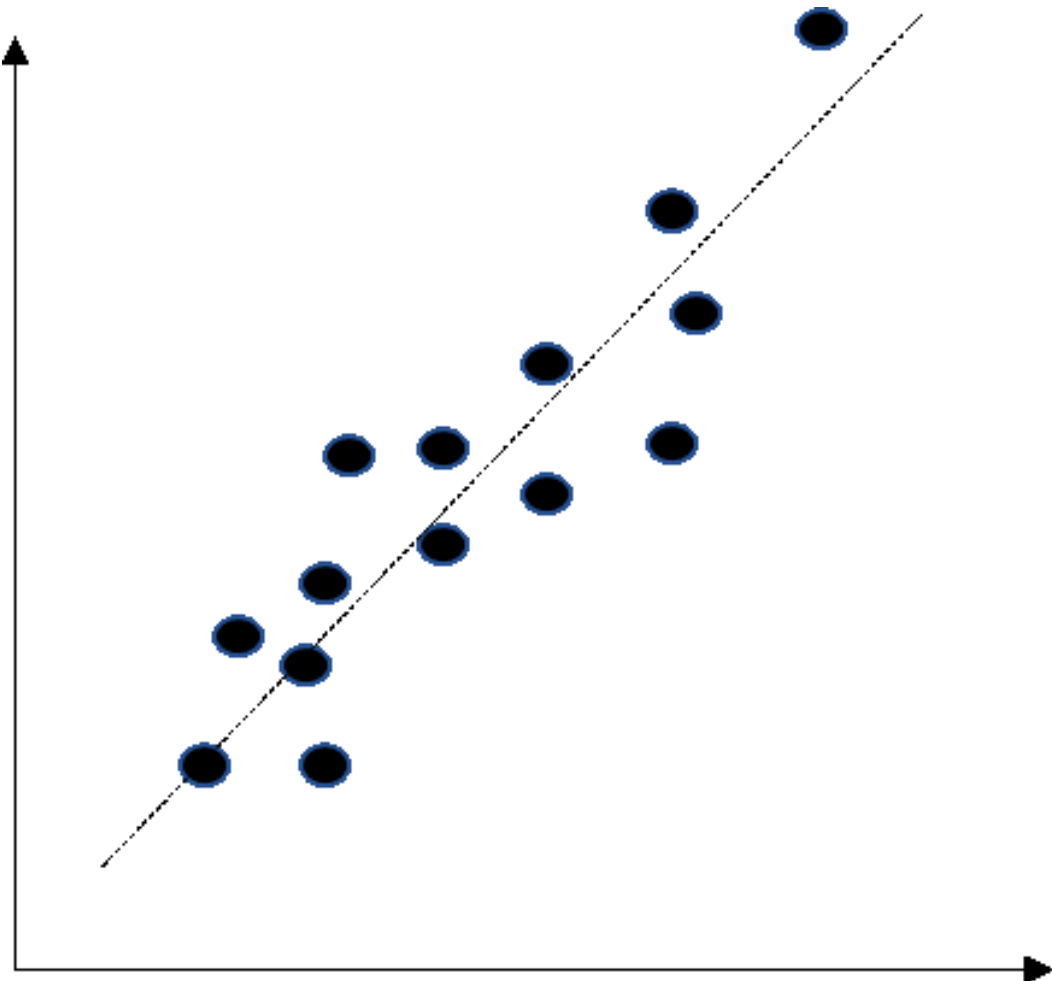
Построение признаков: PCA



Снижение размерности

Цель: построить меньшее количество новых признаков, которые **содержат максимум информации из исходных**

- Линейные методы (PCA)
- Нелинейные методы (MDS, TSNE, UMAP)



Вспомним переход к новым координатам

- Пусть у нас есть объект x с тремя признаками: $x = (-0.343, -0.754, 0.241)$
- Можно сказать, что он представлен в пространстве, натянутом на три стандартных базисных вектора:

$$e_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad e_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad e_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$x = -0.343e_1 + -0.754e_2 + 0.241e_3$$

- Предположим, мы хотим перейти к другому базису, например:

$$w_1 = \begin{pmatrix} -0.390 \\ 0.089 \\ -0.916 \end{pmatrix} \quad w_2 = \begin{pmatrix} -0.639 \\ -0.742 \\ 0.200 \end{pmatrix} \quad w_3 = \begin{pmatrix} -0.663 \\ 0.664 \\ 0.346 \end{pmatrix}$$

- Как спроецировать наш объект x из исходного базиса в данный?

Вспомним переход к новым координатам

- Проецируем:

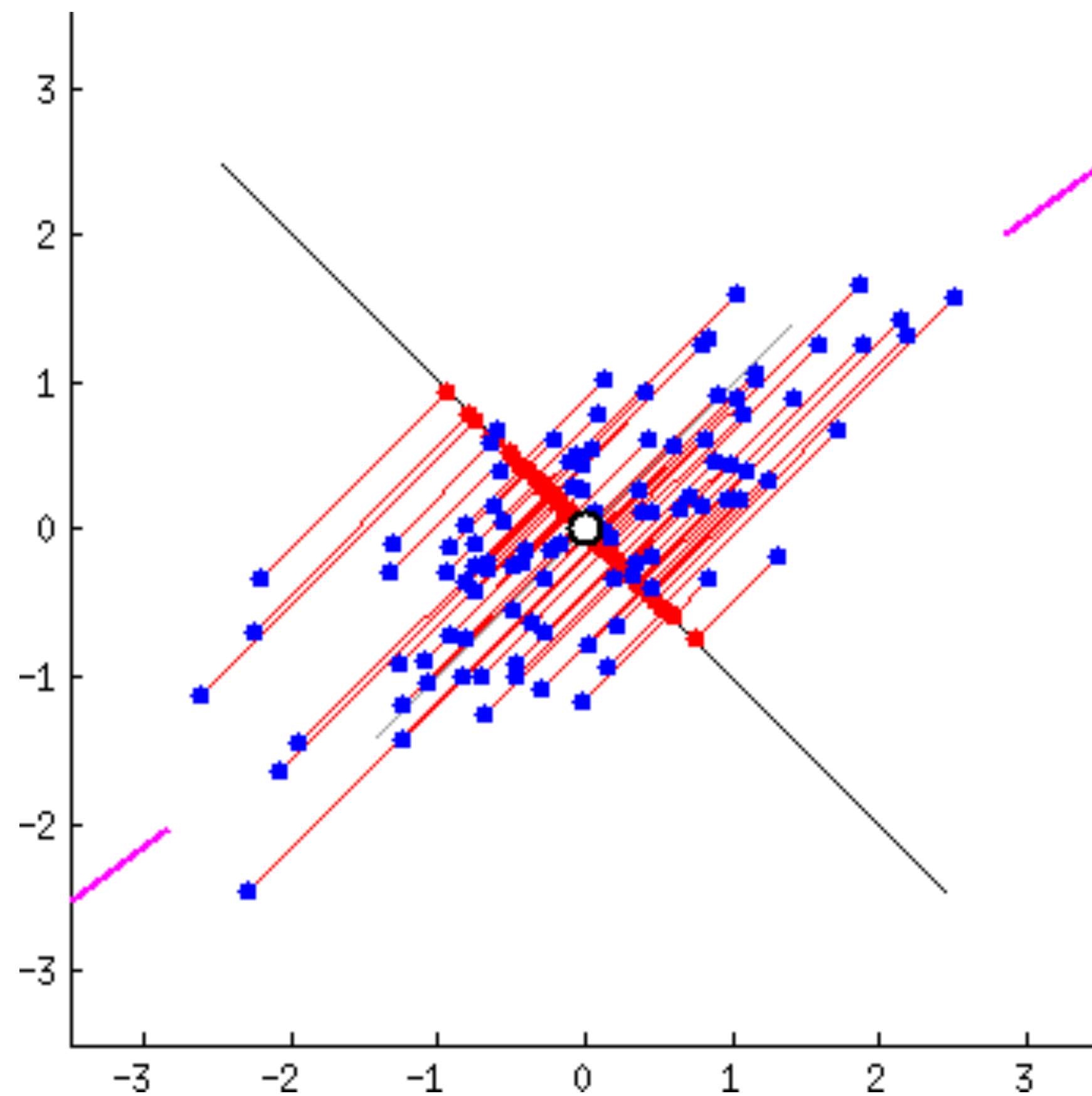
$$w_1 = \begin{pmatrix} -0.390 \\ 0.089 \\ -0.916 \end{pmatrix} \quad w_2 = \begin{pmatrix} -0.639 \\ -0.742 \\ 0.200 \end{pmatrix} \quad w_3 = \begin{pmatrix} -0.663 \\ 0.664 \\ 0.346 \end{pmatrix}$$

$$z = W^{\top} x = \begin{pmatrix} -0.390 & 0.089 & -0.916 \\ -0.639 & -0.742 & 0.200 \\ -0.663 & 0.664 & 0.346 \end{pmatrix} \begin{pmatrix} -0.343 \\ -0.754 \\ 0.241 \end{pmatrix} = \begin{pmatrix} -1.154 \\ 0.828 \\ 0.190 \end{pmatrix}$$

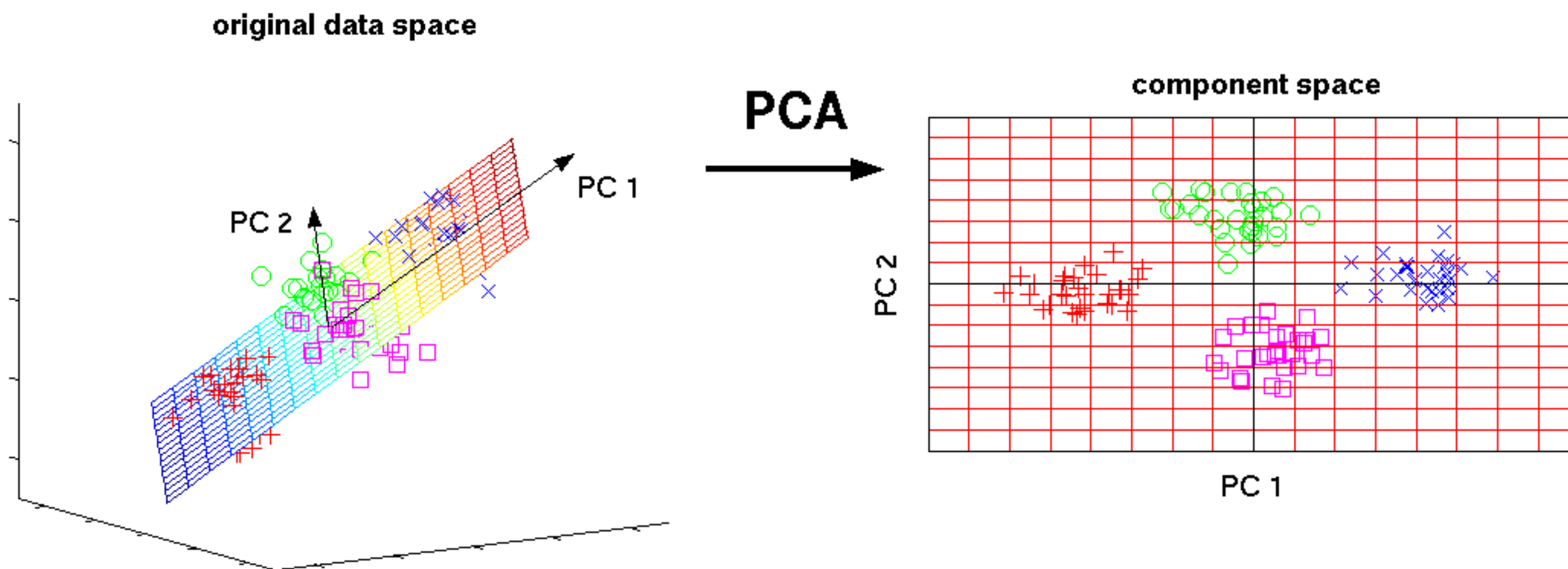
- То есть:

$$x = -1.154w_1 + 0.828w_2 + 0.190w_3$$

РСА. Интуиция



РСА. Интуиция



РСА. В двух словах

Совершаем переход к новым осям так, что:

- новые переменные являются линейной комбинацией старых переменных;
- новые оси ортогональны друг другу;
- дисперсия вдоль новых осей максимальная.

РСА. Находим главные компоненты

- Новые оси ортогональны друг другу:

$$\langle w_i, w_j \rangle = \begin{cases} 1, & i = j \\ 0 & i \neq j \end{cases}$$

- Дисперсия вдоль новых осей максимальна (предполагаем, что исходные признаки **центрированы**):

$$\begin{aligned} \sigma_w^2 &= \frac{1}{n-1} \sum_{i=1}^n (z_i - \mu_z)^2 = \frac{1}{n-1} \sum_{i=1}^n (w^\top x_i)^2 = \\ &= \frac{1}{n-1} \sum_{i=1}^n w^\top (x_i x_i^\top) w = w^\top \left(\frac{1}{n-1} \sum_{i=1}^n x_i x_i^\top \right) w = \\ &= w^\top C w \rightarrow = w^\top \frac{1}{n-1} X^\top X w \rightarrow \max_w \end{aligned}$$

РСА. Находим главные компоненты

- $C = \frac{1}{n-1}X^{\top}X$ — ковариационная матрица
- [Interpreting Covariance Matrix](#)
- То есть, чтобы найти первую главную компоненту, надо решить такую задачу:

$$\begin{cases} w_1^{\top} C w_1 \rightarrow \max_{w_1} \\ w_1^{\top} w_1 = 1 \end{cases}$$

РСА. Решение для первой компоненты

- Изначально было так:

$$w_1^T C w_1 \rightarrow \max_{w_1}$$

- Строим лагранжиан:

$$\mathcal{L}(w_1, \nu) = w_1^T C w_1 - \nu(w_1^T w_1 - 1) \rightarrow \max_{w_1, \nu}$$

- Считаем производную по w_1 :

$$\frac{\partial \mathcal{L}}{\partial w_1} = C w_1 - \nu w_1 = 0 \Leftrightarrow C w_1 = \nu w_1$$

- Подставим одно в другое:

$$w_1^T C w_1 = \nu w_1^T w_1 = \nu \rightarrow \max$$

Задача о собственном векторе, собственном числе матрицы

Оказывается, что у матрицы ковариаций C :

- Все собственные числа $\lambda_i \in \mathbb{R}, \lambda_i \geq 0$ (упорядочим индексы по возрастанию так, что $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$).
- Собственные вектора при $\lambda_i \neq \lambda_j$ ортогональны: $v_i^\top v_j = 0$.
- Для уникальных λ_i уникальны и v_i .

РСА. Решение для первой компоненты

- Получили следующее:

$$w_1^T C w_1 = \nu w_1^T w_1 = \nu \rightarrow \max$$

- Это значит, что:
 - ν — наибольшее собственное число матрицы C , а именно — λ_1 .
 - w_1 — собственный вектор при λ_1 .

РСА. Задача и решение для второй компоненты

$$\begin{cases} w_2^\top C w_2 \rightarrow \max_{w_2} \\ w_2^\top w_2 = 1 \\ w_2^\top w_1 = 0 \end{cases}$$

- При помощи аналогичных операций мы придём к тому, что w_2 — собственный вектор при втором по величине собственном числе матрицы C .
- То есть, поиск главных компонент сводится к поиску собственных векторов и собственных чисел матрицы C !

Как выбрать количество новых признаков

- Из предыдущих слайдов мы помним, что

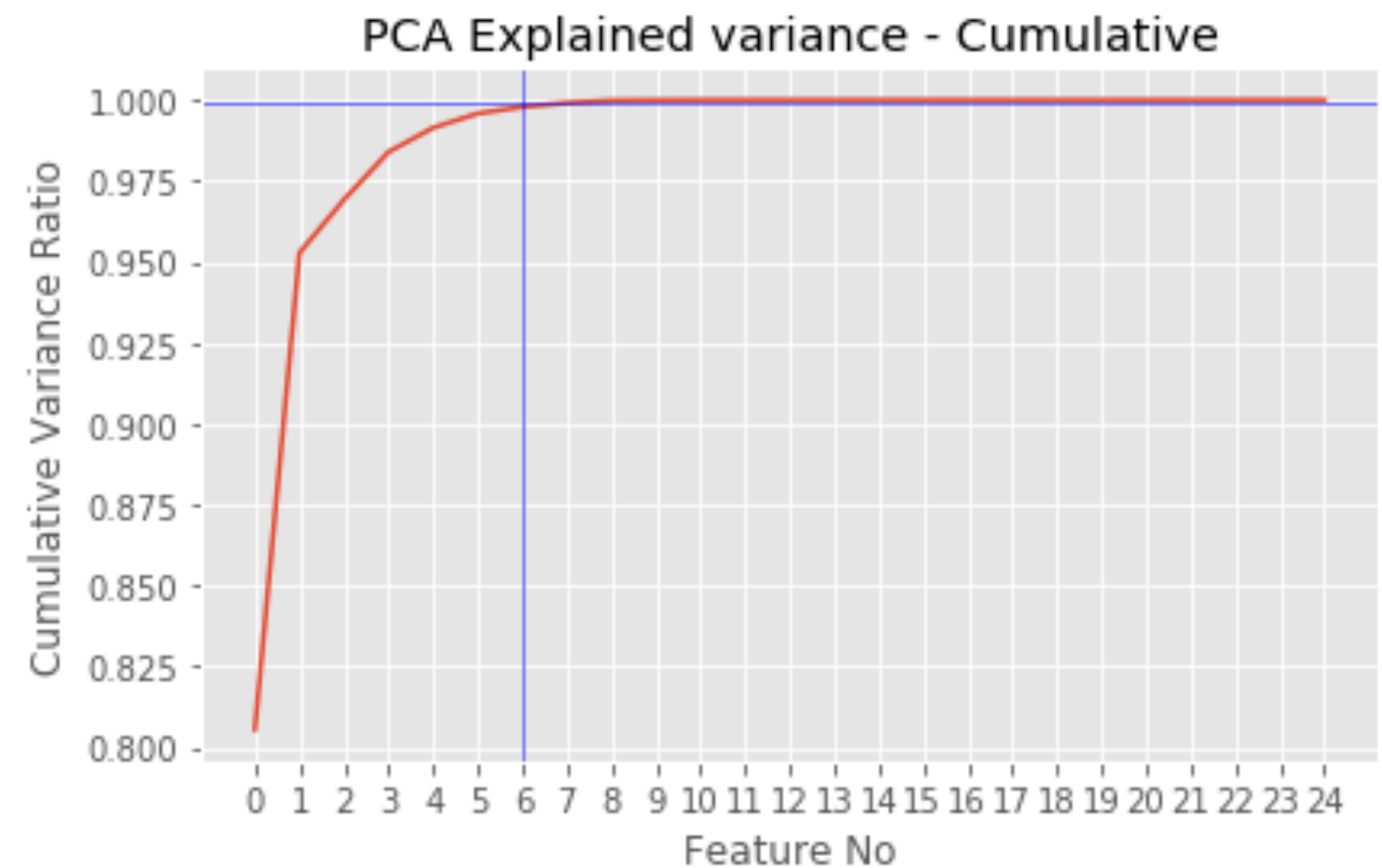
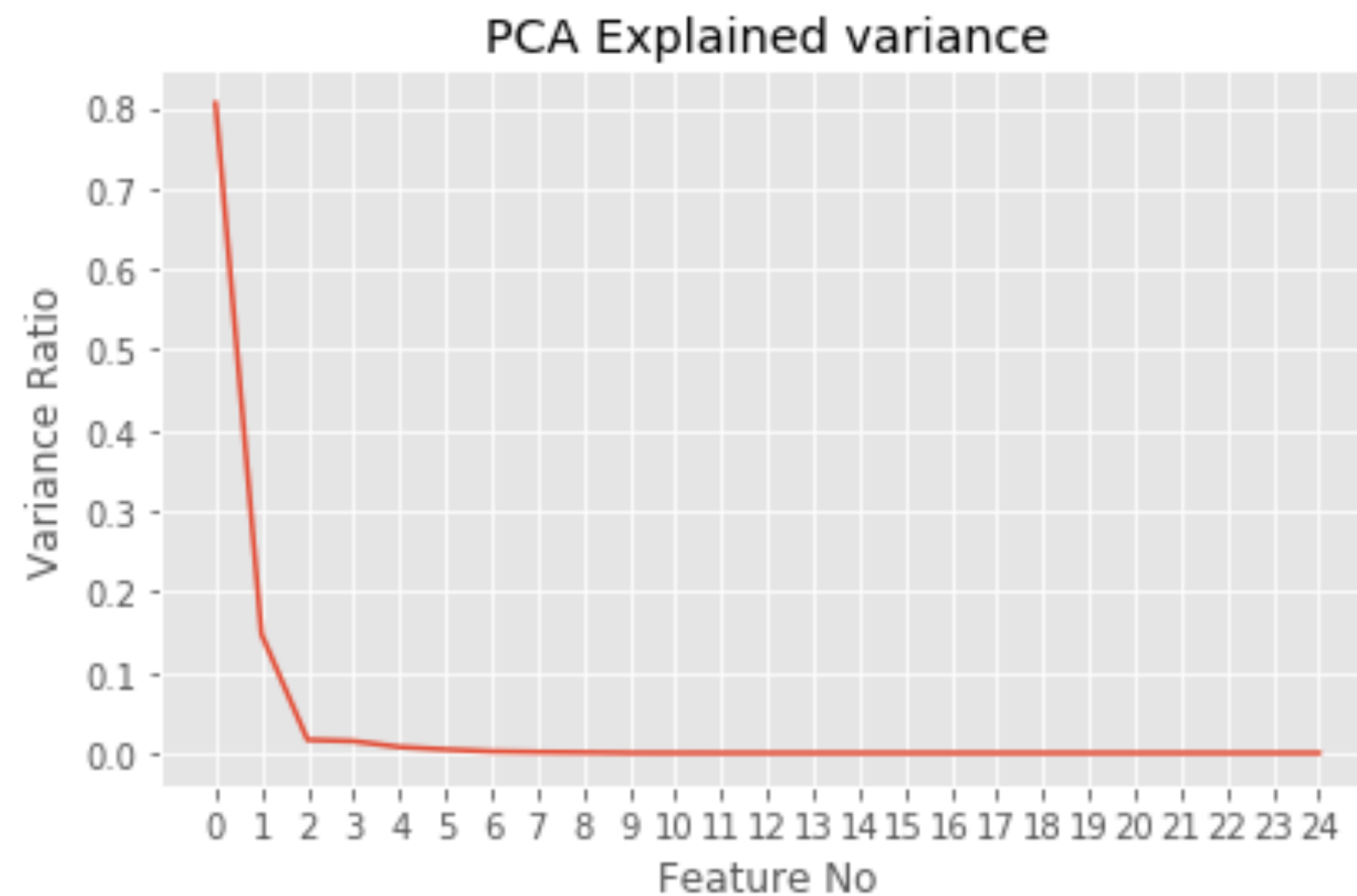
$$w_1^T C w_1 = \lambda_1.$$

- То есть, λ_1 равна дисперсии точек, спроецированных на первую базовую компоненту.
- Величина

$$\frac{\lambda_i}{\sum_{d=1}^D \lambda_d}$$

показывает долю объяснённой дисперсии компоненты i .

Как выбрать количество новых признаков



Построение признаков: SVD

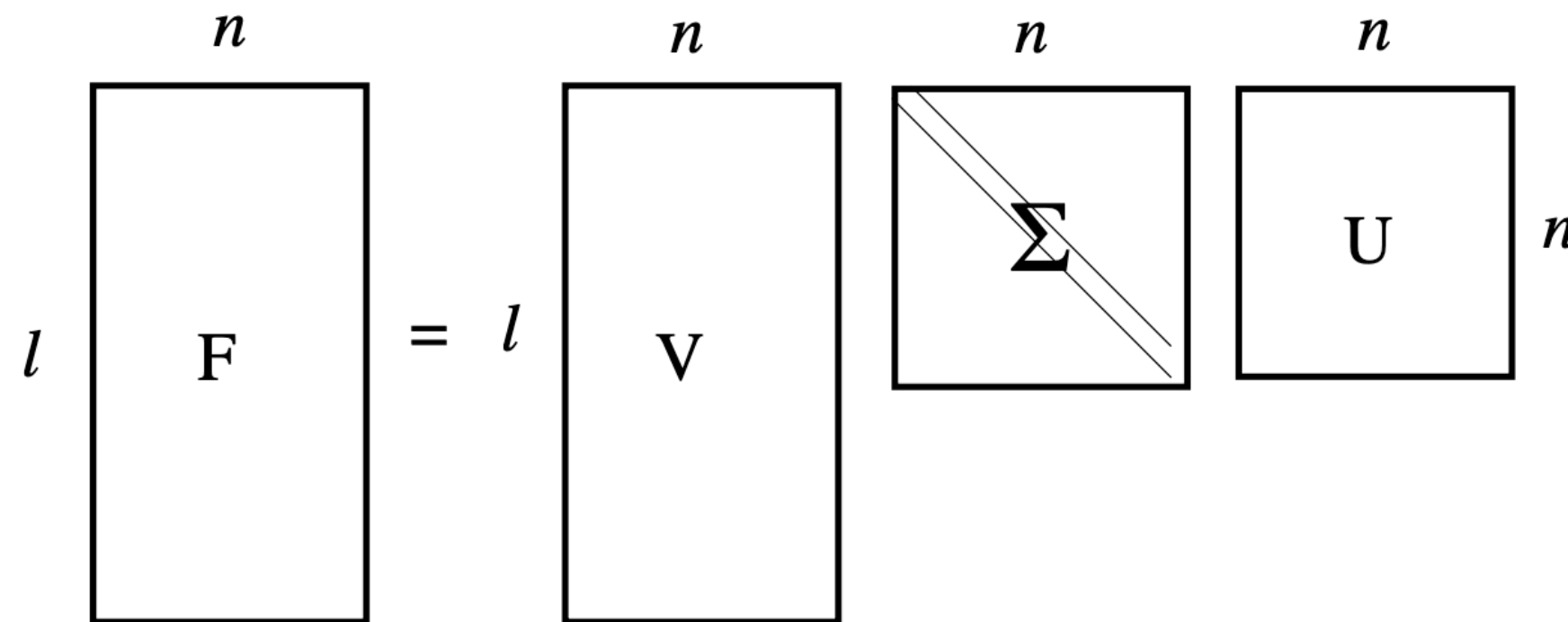


Singular Value Decomposition (SVD)

Любая вещественная матрица $F_{l,n}$ разложима следующим образом:

$$F_{l,n} = V_{l,n} * \Sigma_{n,n} * U_{n,n}^T$$

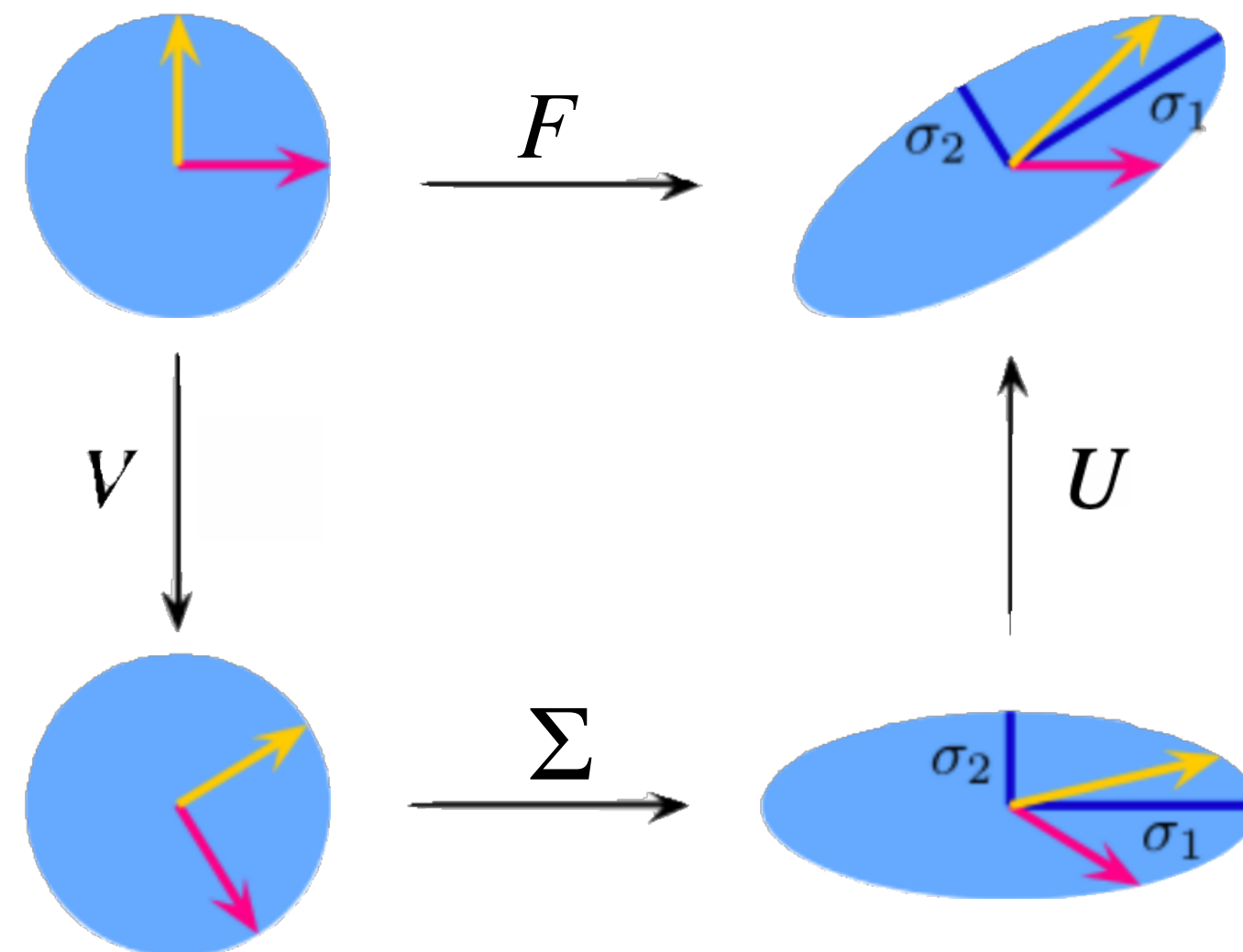
где V и U - ортогональные матрицы (состоят из левых и правых сингулярных векторов), Σ - на главной диагонали сингулярные числа



SVD. Геометрическое объяснение

Пусть исходная матрица F задает некоторое линейное преобразование исходного пространства.

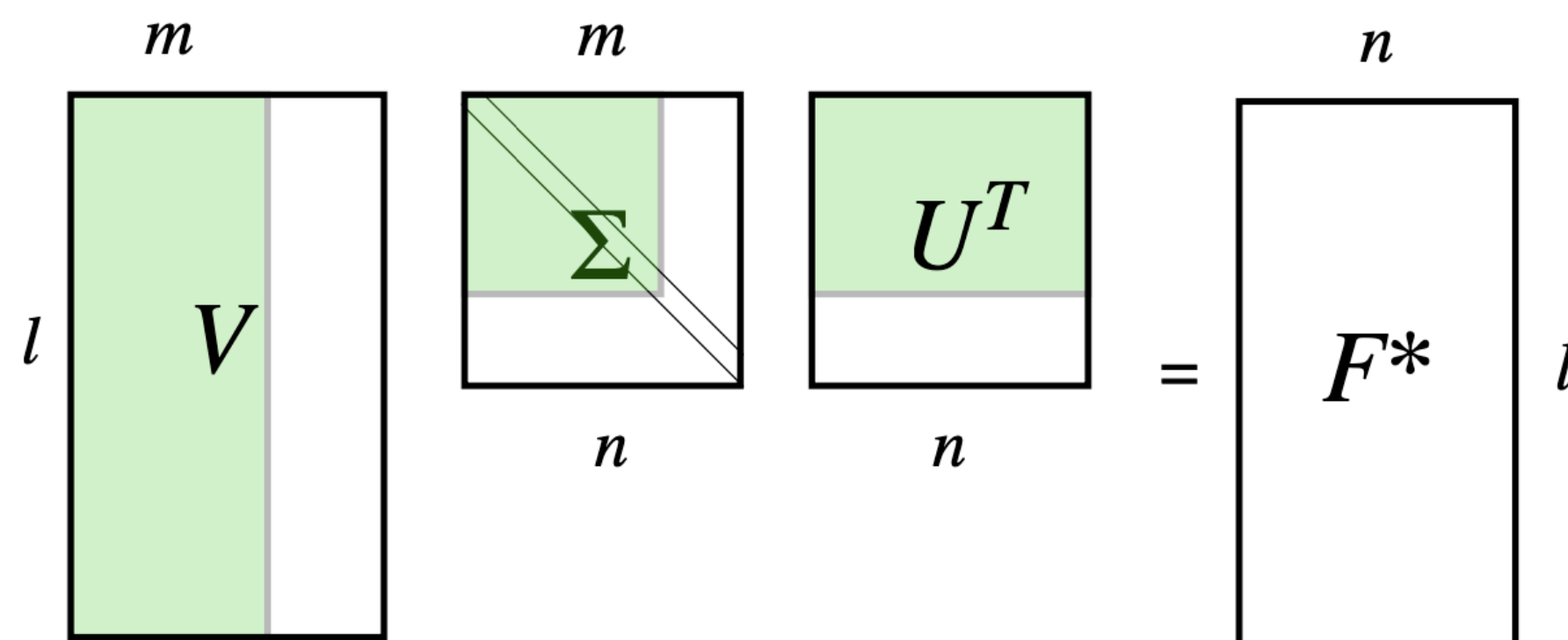
Тогда это линейное преобразование можно разбить на последовательность более простых линейных преобразований



Truncated SVD

Можно взять m строк матрицы Σ , соответствующие наибольшим сингулярным числам, m столбцов матриц V и $U \rightarrow$ получить приближенное представление матрицы F

$$F^*_{l,n} = V_{l,m} * \Sigma_{m,m} * U^T_{m,n}$$



Связь PCA и SVD

- Пусть всё так же X центрировано, $C = \frac{1}{n-1}X^T X$ — ковариационная матрица.
- Так как C — симметричная матрица, то она может быть представлена в виде $C = VL V^T$, где V — это матрица с собственными векторами C , а L — диагональная матрица с собственными значениями по убыванию.
- Сингулярное разложение позволяет нам представить произвольную матрицу в виде $X = USV^T$, где U, V — унитарные матрицы, S — диагональная матрица с сингулярными значениями по убыванию.

Связь PCA и SVD

- Подставим теперь разложение X , полученное через SVD, в определение C :

$$C = \frac{1}{n-1} V S U^T U S V^T = V \frac{S^2}{n-1} V^T$$

- Теперь осталось только сравнить это выражение для C с её представлением через собственные векторы.
- Легко видеть, что V — это и есть собственные векторы матрицы ковариаций, а собственные её значения связаны с сингулярными значениями как

$$\lambda_i = s_i^2 / (n - 1).$$

PCA, SVD. Recap

- PCA ищет такую гиперплоскость, до которой будет минимально суммарное расстояние от точек выборки.
- PCA ищет такие ортогональные проекции, дисперсия вдоль которых для точек выборки будет максимальна.
- PCA строит такой базис, в котором новые признаки ортогональны.
- Можно применять в моделях и для визуализации.
- [Tutorial 1](#)
- [PCA SVD tutorial](#)
- [Относительно лёгкая и подробная статья про связь PCA, SVD](#)

Нелинейные преобразования признаков: Manifold Learning



Manifold learning

РСА - линейный метод снижения размерности признакового пространства
Но никто нас не ограничивает в выборе преобразования.

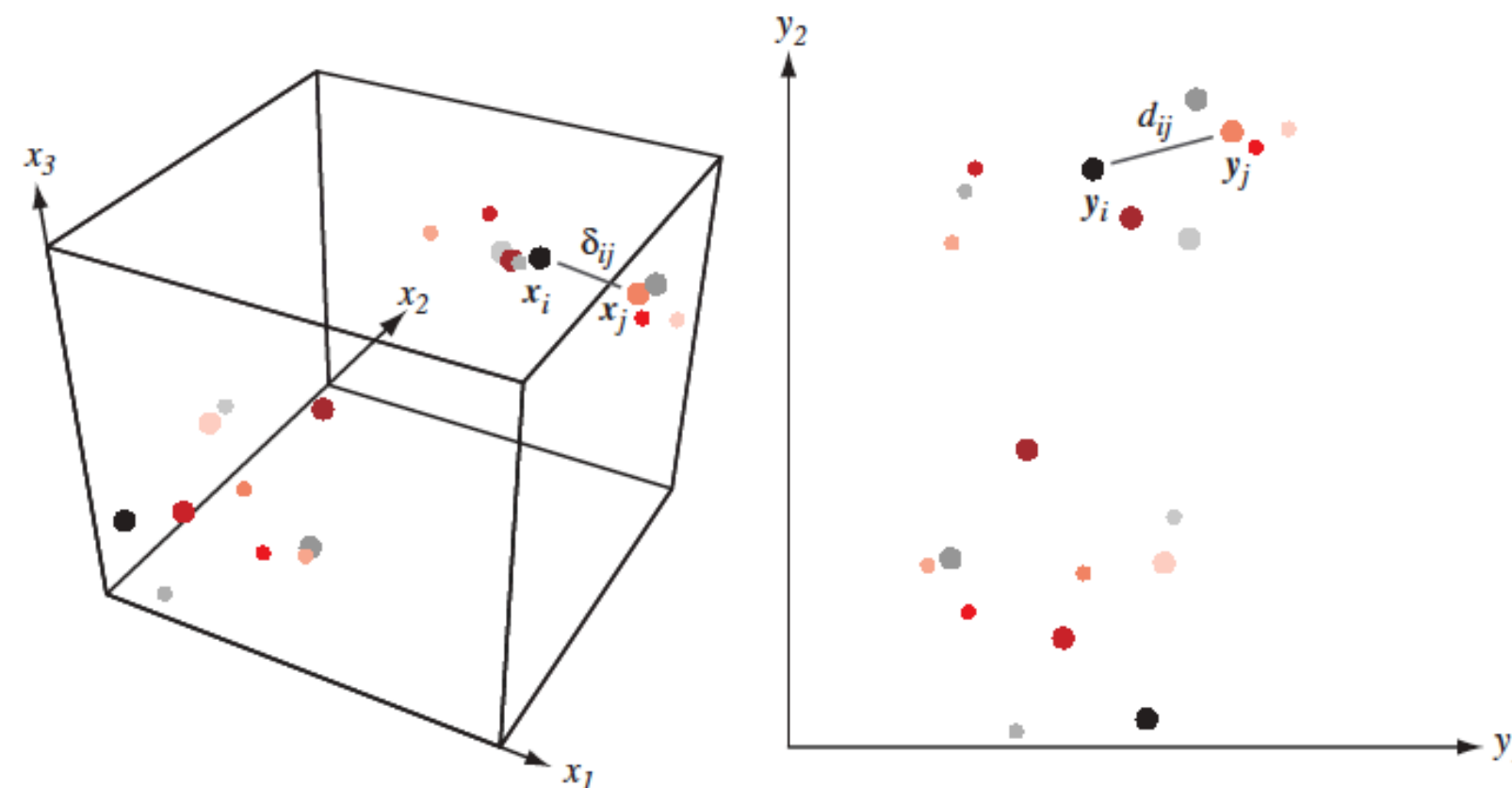
Есть ряд **нелинейных** методов снижения размерности: Isomap, t-SNE, MDS, UMAP

Кроме того, с РСА отлично работает kernel trick.

Multidimensional Scaling (MDS)

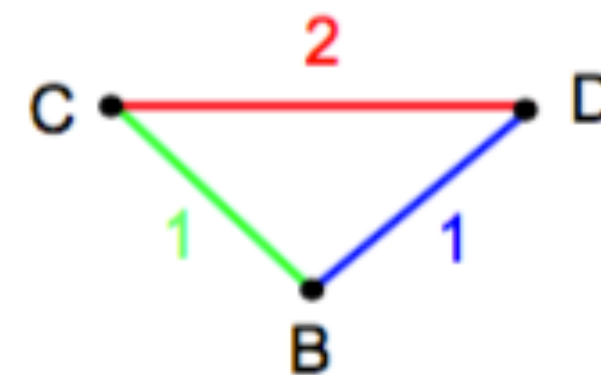
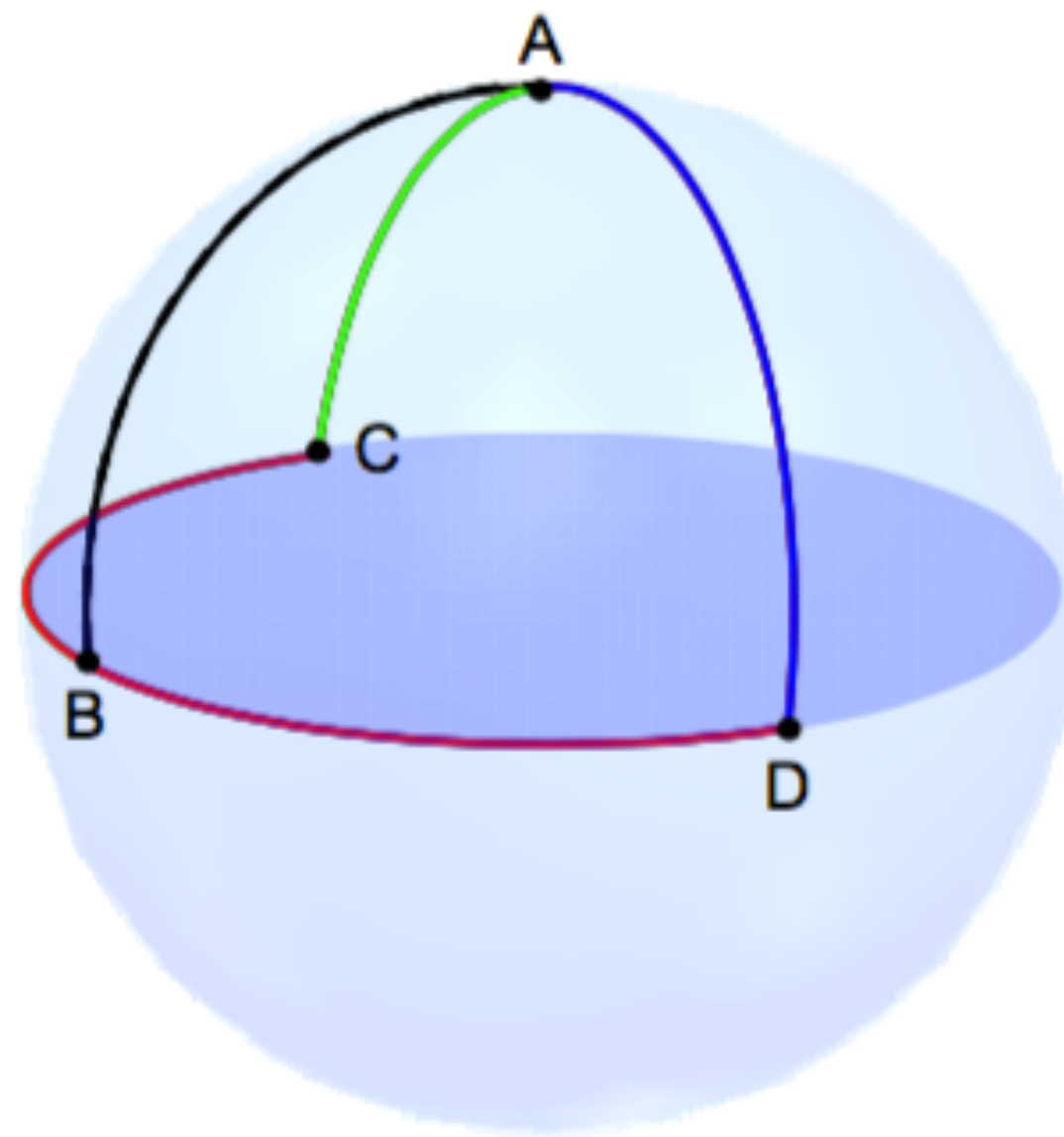
Идея:

- Перейти в пространство меньшей размерности так, чтобы расстояния между объектами в новом пространстве были подобны расстояниям в исходном пространстве.
- Дано $X = [x_1, \dots, x_n] \in \mathbb{R}^{N \times D}$ и/или δ_{ij} — мера близости между (x_i, x_j) .
- Надо найти $Y = [y_1, \dots, y_n] \in \mathbb{R}^{N \times d}$ такие, что $\delta_{ij} \approx d(y_i, y_j) = \|y_i - y_j\|^2$.

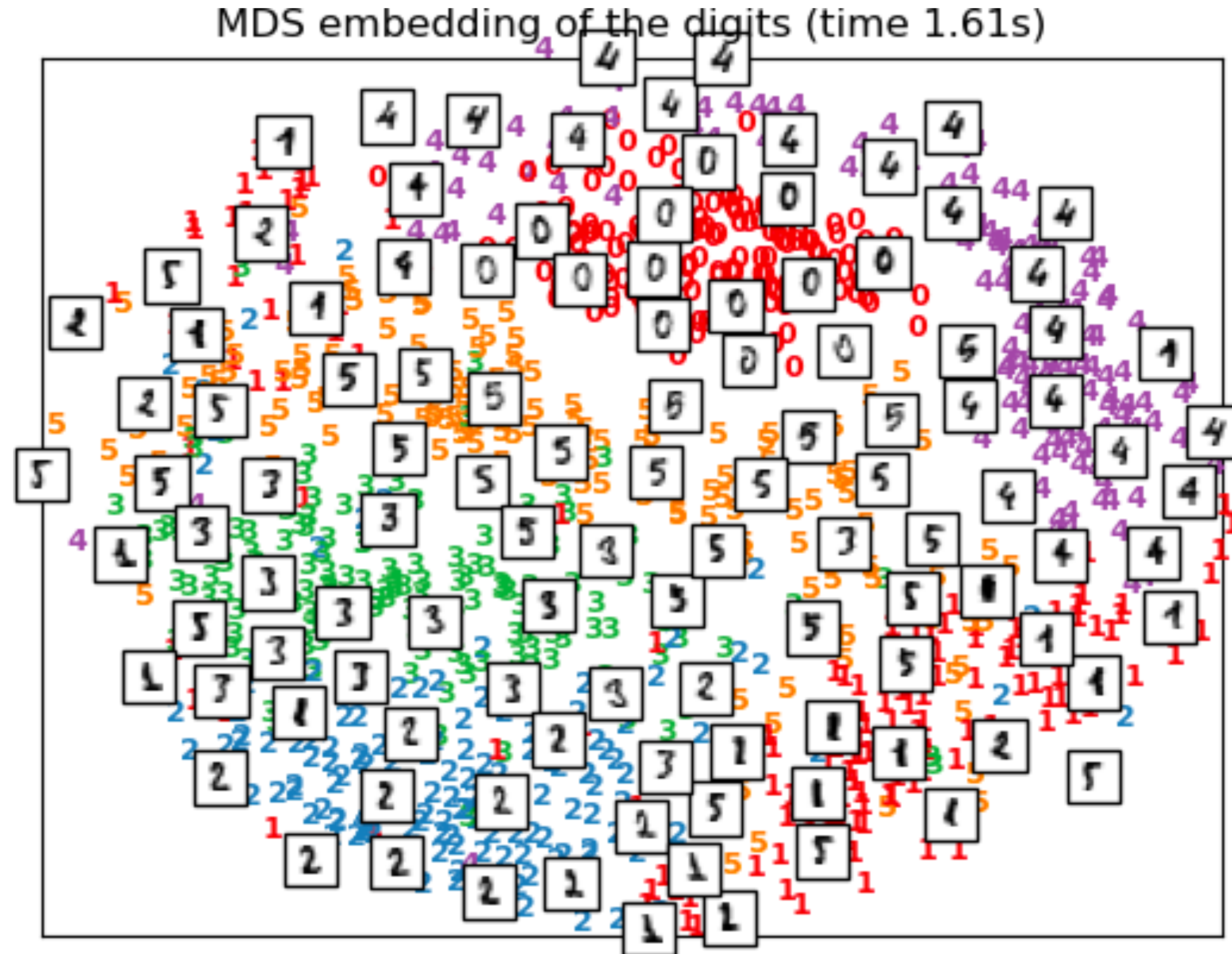


Multidimensional Scaling (MDS)

Понятно, что точно воспроизвести расстояния получится не всегда:



Multidimensional Scaling (MDS)



t-distributed Stochastic Neighbour Embedding (t-SNE)

- t-SNE — практически многомерное шкалирование.
- Вместо этого мы попытаемся перенести «окрестность» точек из исходного пространства в пространство меньшей размерности.
- Полученные расстояния, скорее всего, не будут соотноситься с исходными.

t-distributed Stochastic Neighbour Embedding (t-SNE)

- Схожесть между объектами в исходном пространстве $\mathbb{R}^m$

$$p(i, j) = \frac{p(i | j) + p(j | i)}{2n}, \quad p(j | i) = \frac{\exp(-\|\mathbf{x}_j - \mathbf{x}_i\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|\mathbf{x}_k - \mathbf{x}_i\|^2 / 2\sigma_i^2)}$$

- σ_i неявно задаётся пользователем через параметр perplexity:

$$Perp(P_i) = 2^{H(P_i)},$$

$$H(P_i) = - \sum_j p(j | i) \log_2 p(j | i).$$

t-distributed Stochastic Neighbour Embedding (t-SNE)

- Схожесть между объектами в целевом пространстве $\mathbb{R}^k, k \ll m$

$$q(i, j) = \frac{g(\|\mathbf{y}_i - \mathbf{y}_j\|)}{\sum_{k \neq l} g(\|\mathbf{y}_i - \mathbf{y}_j\|)},$$

где $g(z) = \frac{1}{1 + z^2}$ — распределение Коши (t-распределение Стьюдента с одной степенью свободы).

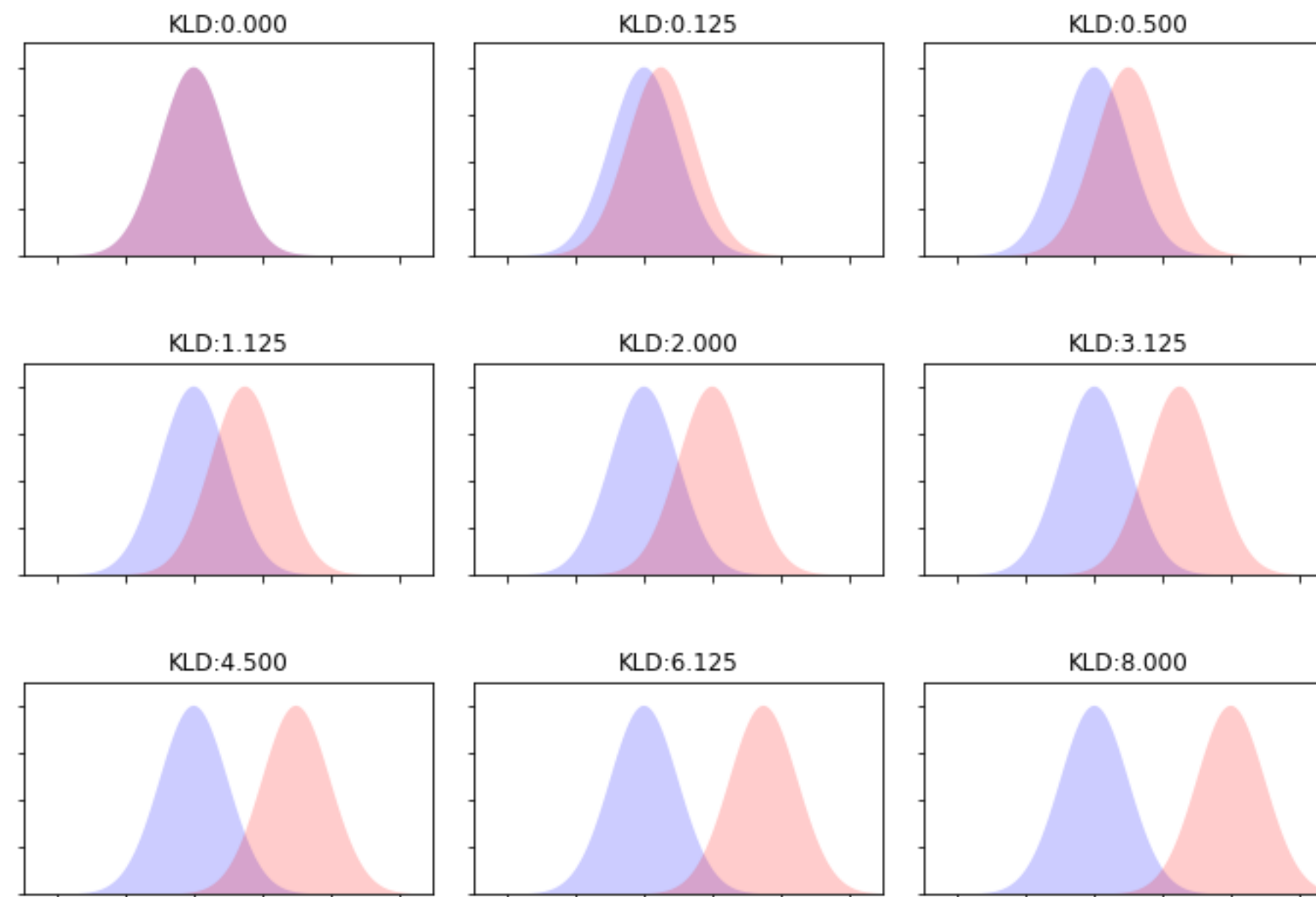
- Критерий

$$J_{t-SNE}(\mathbf{y}) = KL(P \| Q) = \sum_i \sum_j p(i, j) \log \frac{p(i, j)}{q(i, j)} \rightarrow \min_{\mathbf{y}}.$$

Дивергенция Кульбака–Лейблера

- Насколько распределение P отличается от распределения Q ?

$$KL(P\|Q) = \sum_z P(z) \log \frac{P(z)}{Q(z)}.$$



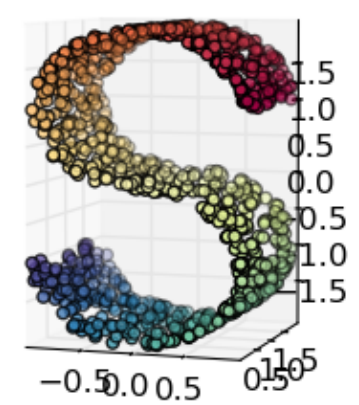
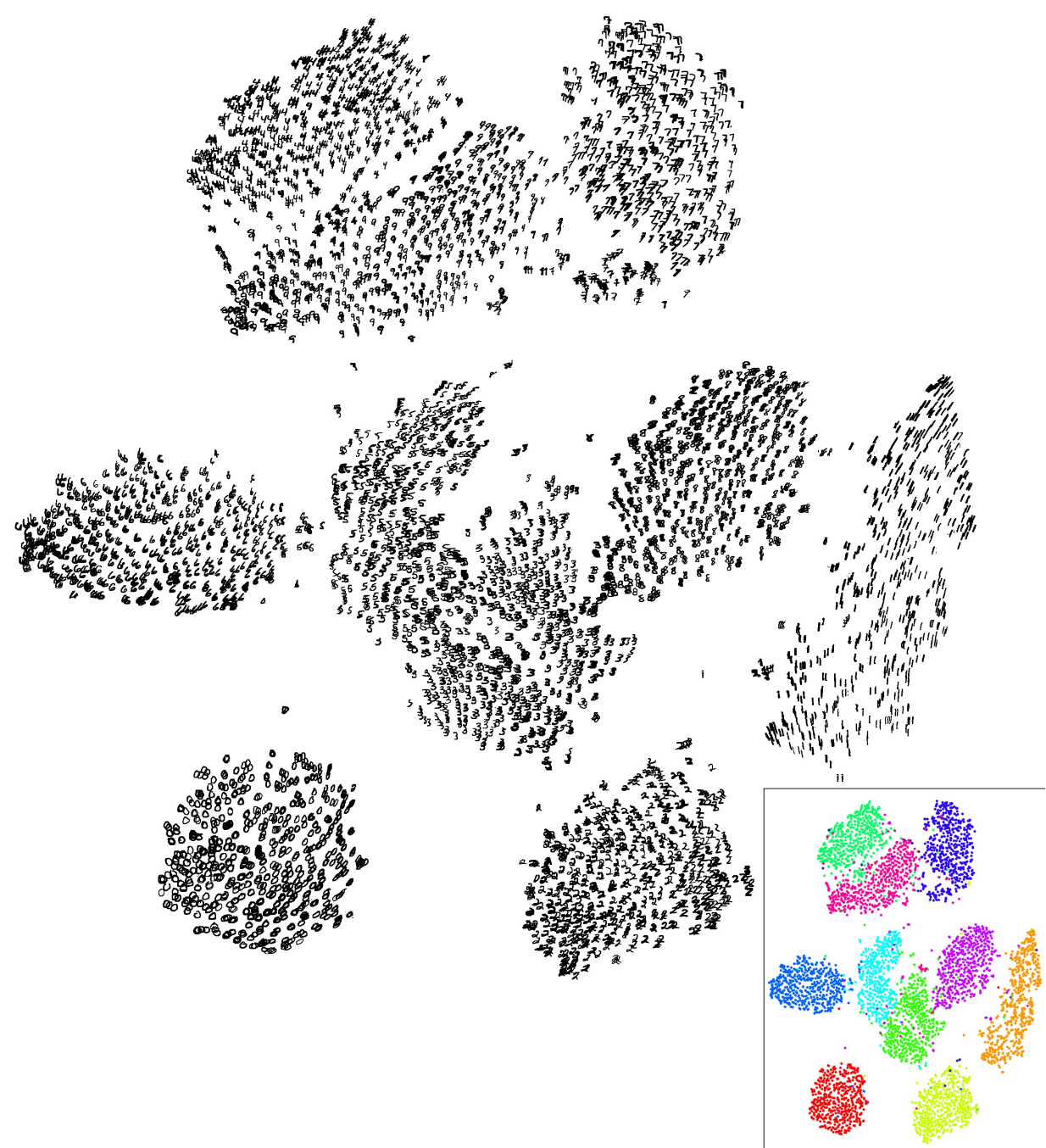
t-SNE. Оптимизация

- Оптимизируем $J_{t-SNE}(y)$ с помощью градиентного спуска

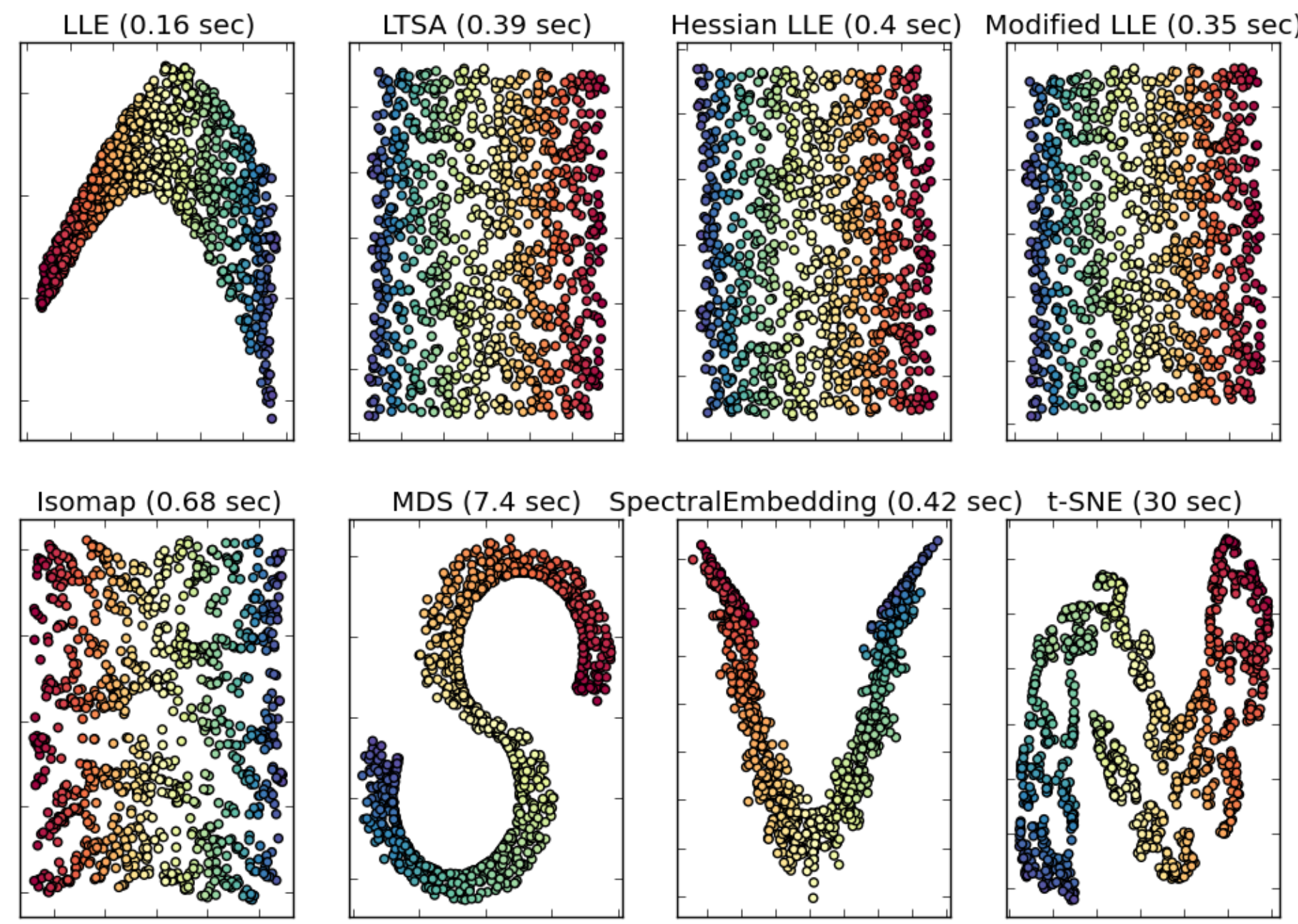
$$\frac{\partial J_{t-SNE}}{\partial y_i} = 4 \sum_j (p(i, j) - q(i, j))(y_i - y_j)g(|y_i - y_j|).$$

- [Статья](#)
- [Примеры](#)
- [Демо и советы](#)
- t-SNE может быть нестабильным
- Размеры полученных сгустков могут ничего не значить
- Расстояния между кластерами могут ничего не значить
- Полностью шумовые данные могут выдать структуру

t-SNE. Визуализация



Manifold Learning with 1000 points, 10 neighbors



Uniform Manifold Approximation and Projection for Dimension Reduction

- UMAP — развитие подхода t-SNE.
- Более быстрый за счёт отсутствия нормализации для исходного и нового пространств.
- Старается сохранить не только локальную, но и глобальную структуру в данных.

UMAP

- Схожесть между объектами в исходном пространстве $\mathbb{R}^m$

$$p_{ij} = p(i, j) = p(i | j) + p(j | i) - p(i | j)p(j | i), \quad p(i | j) = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right)$$

- $d(x_i, x_j)$ — расстояние между точками (необязательно евклидово), ρ_i — расстояние от i -го элемента до ближайшего соседа, σ_i ищутся, исходя из уравнения (k — гиперпараметр, число соседей):

$$\sum_j p(i, j) = \log_2 k$$

UMAP

- Схожесть между объектами в целевом пространстве $\mathbb{R}^k, k < m$

$$q_{ij} = q(i, j) = (1 + a(y_i - y_j)^{2b})^{-1},$$

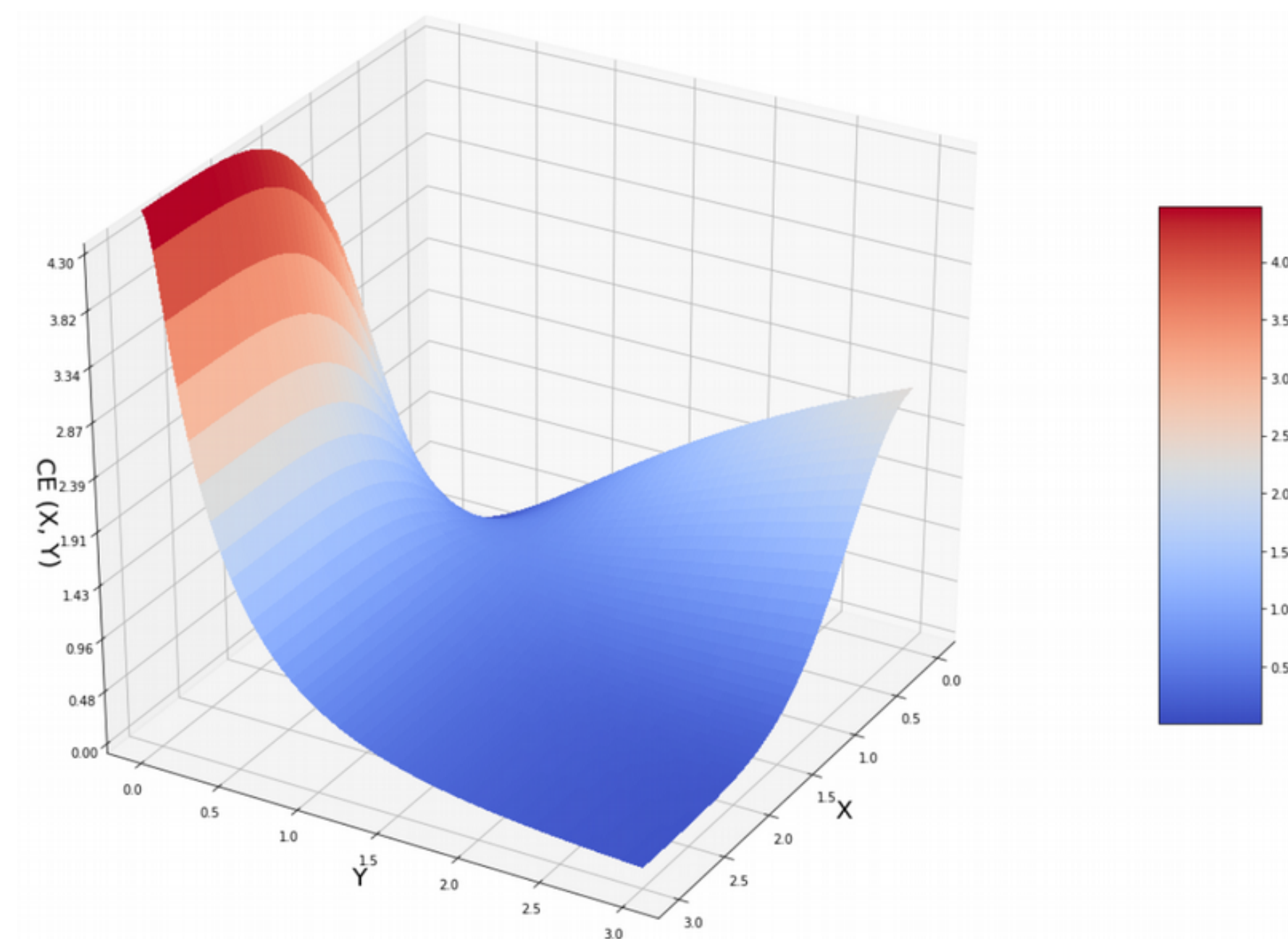
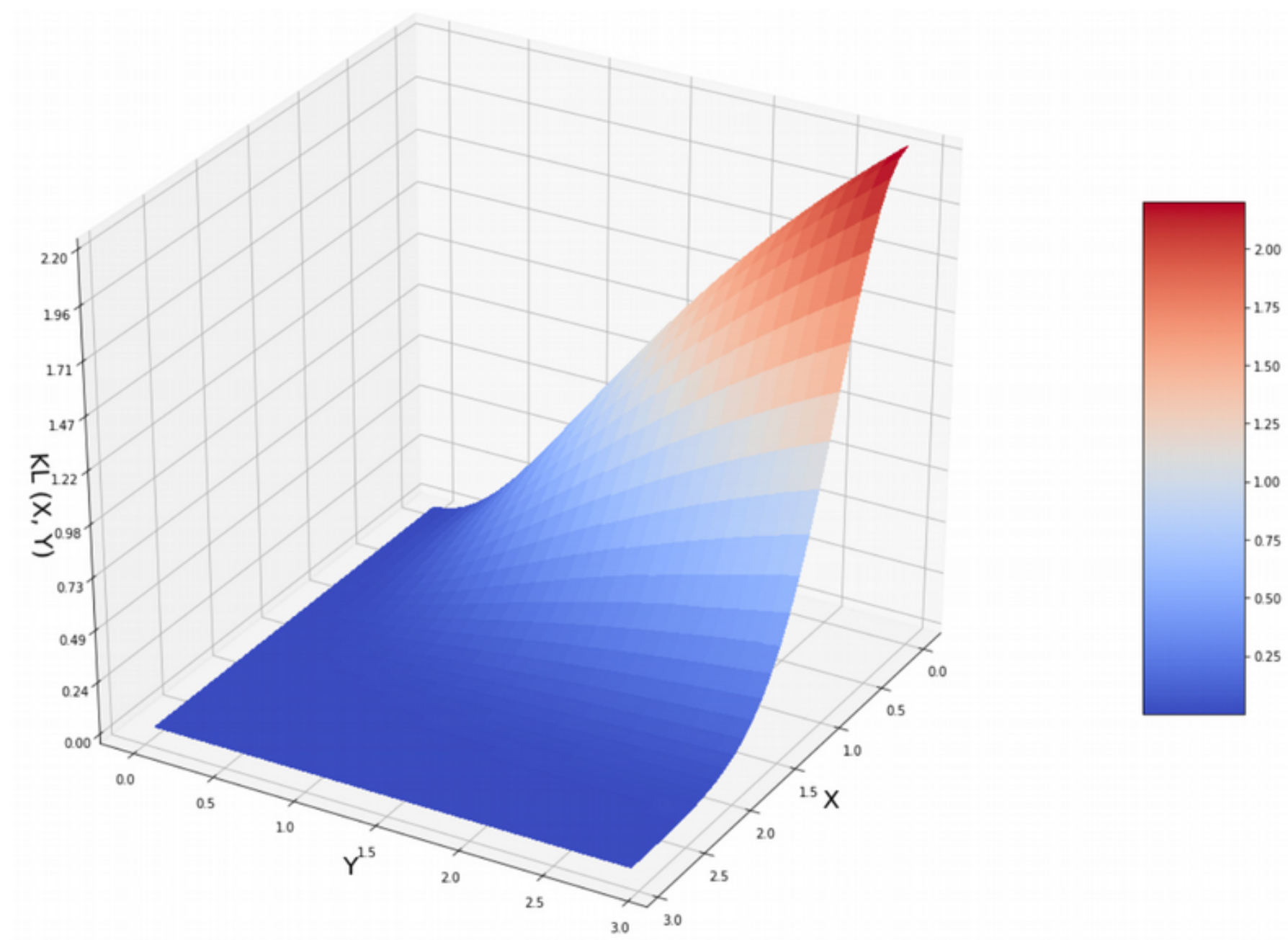
где $a \approx 1.93$ и $b \approx 0.79$ — гиперпараметры и их значения по умолчанию.

- Критерий — fuzzy set cross entropy, кросс-энтропия для нечётких множеств

$$CE(X, Y) = \sum_i \sum_j \left[p_{ij}(X) \log \frac{p_{ij}(X)}{q_{ij}(Y)} - (1 - p_{ij}(X)) \log \frac{(1 - p_{ij}(X))}{(1 - q_{ij}(Y))} \right]$$

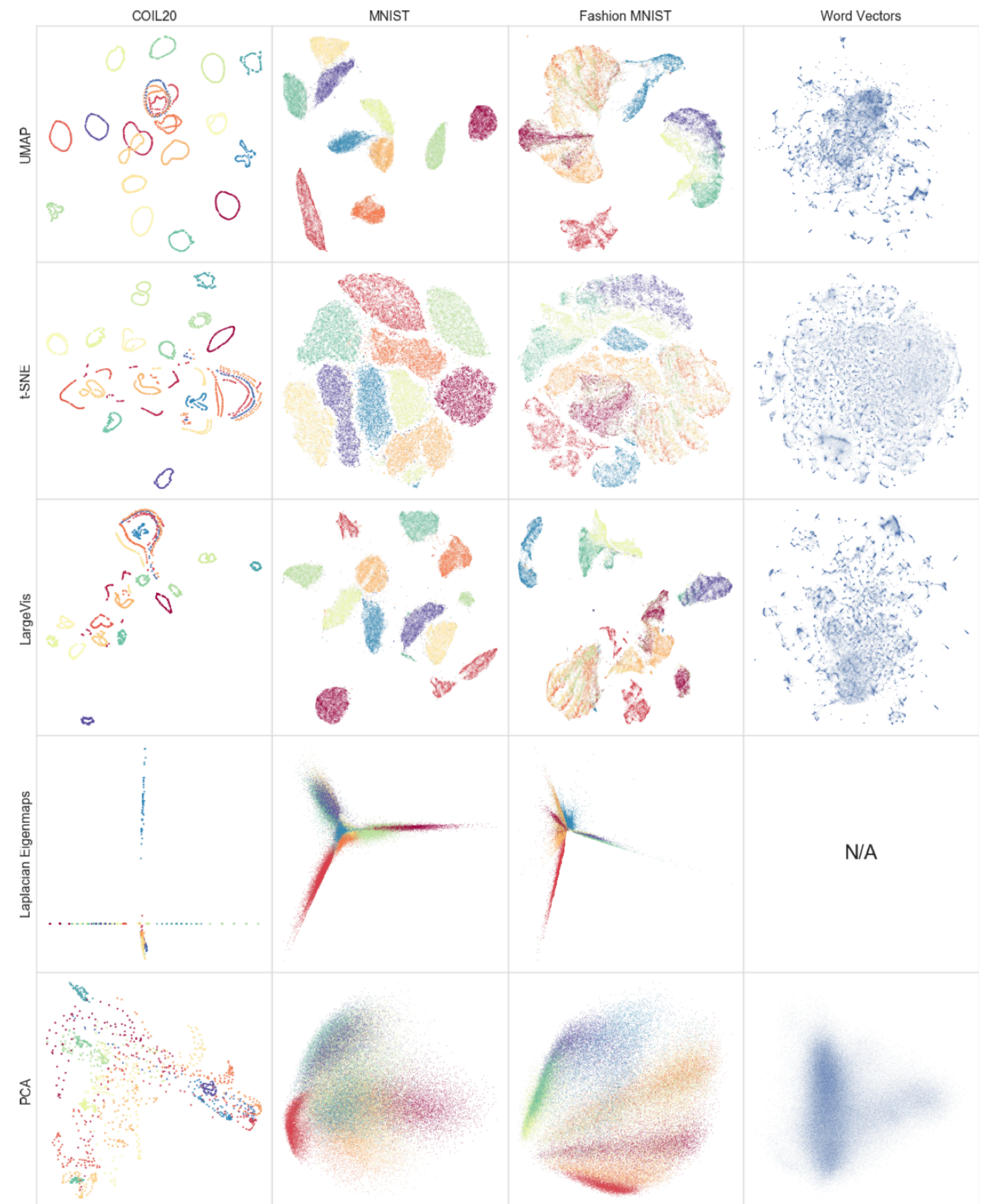
- Оптимизируемся с помощью стохастического градиентного спуска.

t-SNE vs UMAP: критерии



UMAP

- [Оригинальная статья](#)
- [Статья со сравнением t-SNE и UMAP, с очень хорошей визуализацией](#)
- [Статья про UMAP](#)



Thanks for attention!